Author validation task guide

Shahriar Hasan
Md. Ashikul Islam

npm supply-chain research / EMSE manuscript

Your deliverables
Two independent 120-row review sheets; documented resolution of disagreements; actual contribution descriptions; and each author's approval of the final manuscript.

Start here: read Sections 1–7, open Evidence_Index.html, then work independently in your own CSV. Do not compare initial decisions until both passes are complete and saved.

Prepared 28 October 2026, Asia/Dhaka.
This document gives instructions. It does not record completed reviews or author approval. The endpoint audit is additional validation, not a journal-mandated procedure.

Contents

1 What you are trying to finish **2**

2 Open these files first **2**

3 Who does what **3**

4 Make a realistic schedule **3**

5 Understand the evidence before assigning labels **3**

6 Use exactly these three label values **4**

7 The five criteria: exact decision rules **4**

7.1 Repository mapping supported 4

7.2 Workflow file present 5

7.3 Installation directive observed 5

7.4 Relevant lockfile present 5

7.5 Dependency/version example supported 6

8 Repeat this workflow for each row **6**

9 Fill the CSV without damaging it **7**

10 Check your own sheet before sharing labels **7**

11 Preserve independent results, then adjudicate **8**

12 Technical handoff and analysis commands **8**

13 What the resulting study may claim **9**

14 Record actual author contributions **9**

15 Read and approve the actual final manuscript **9**

16 Handle the existing journal submission **10**

17 What to return when finished **10**

18 Source and scope **11**

For Shahriar Hasan and Md. Ashikul Islam

Prepared 28 October 2026, Asia/Dhaka. This is a working task guide, not a record that either author has already completed these activities.

Paper: Maintainer Prioritization in npm Supply Chains: Objectives, Representation Sensitivity, and Metadata Limits.

1 What you are trying to finish

There are three separate tasks. First, if you choose to add human endpoint validation, independently inspect the sampled repository evidence and record your actual judgments. Second, describe each author's actual contributions. Third, both authors read and approve the exact manuscript version intended for the journal.

The independent audit is additional research proposed for this study; it is not a specific EMSE requirement. The corrected manuscript currently reports automated indicators and explicitly states that they were not manually validated. Genuine author approval is required for submission. A contribution statement should reflect real work; it cannot be assigned from author order.

Completing this task can strengthen the evidence. It does not guarantee acceptance, validate the whole npm ecosystem, or prove that any workflow actually ran.

Your starting position: 120 sampled repositories; five labels per repository; two separate, unfilled author sheets. Each author makes 600 initial label decisions. That is 1,200 decisions altogether and 600 paired comparisons, with repositories as the sampling units. Some decisions will correctly be unknown.

2 Open these files first

- `Author_Validation_Guide.pdf`: the printable version of this guide.
- `Evidence_Index.html`: open in a browser; use Find to locate an audit ID, such as A001. It lists all 120 repositories, original package/version mappings, and commit-pinned file links. No automated label suggestions are included.
- `Evidence_Packet.md`: the original detailed evidence packet, readable in a Markdown editor.
- `evidence/`: retained registry/GitHub request records and readable source-text copies, linked from the HTML index. These are raw evidence, not automated label decisions.
- `reviews/Shahriar_Hasan_independent_labels.csv`: Shahriar's own sheet.
- `reviews/Md_Ashikul_Islam_independent_labels.csv`: Md. Ashikul's own sheet.
- `templates/review_session_log.csv`: optional log of actual review sessions and access problems.
- `templates/actual_contributions.csv`: blank form for describing actual author contributions.
- `templates/manuscript_approval.md`: checklist to complete only after reading the specified final PDF.
- `validate_reviews.py`: optional local completion checker. It does not decide labels or certify their scientific correctness.

The CSVs in this task folder are working copies. The analysis program reads the canonical files in `manuscript_q1/review_packets/`, not these copies. Finish and preserve your independent copies before handing them back for integration.

3 Who does what

Shahriar reviews all 120 rows in Shahriar's file. Md. Ashikul reviews all 120 rows in Md. Ashikul's file. Do not divide the sample into 60 rows each: that would not create paired independent judgments for the full sample.

Before starting, both authors may discuss the definitions in this guide. For practice, inspect a repository outside the frozen 120-endpoint sample. Practice observations are not study results and must not be entered into the sample sheets.

During the initial pass, do not share decisions, copy each other's notes, consult the automated result table, or ask an assistant to supply the label decisions. You may get help opening files or understanding a YAML keyword. If AI is used to interpret a sampled file or suggest a decision, record that assistance; the pass can no longer be described as wholly unaided manual review.

After both initial sheets are complete and preserved, discuss disagreements. Write the final decision in a separate adjudication file rather than modifying the original labels until they agree.

If you need to clarify a rule after starting, record the clarification and which rows it affects. Preserve the first judgments. Do not silently change the rubric to improve agreement. A substantive change should be reported as a protocol deviation and applied consistently.

4 Make a realistic schedule

Start by timing five real rows independently. They remain part of the actual sample; do not discuss their decisions until both complete initial passes are saved. Use the time you actually spent to estimate the remaining workload.

An illustrative average of 10 to 20 minutes per endpoint implies about 20 to 40 hours per author for 120 endpoints, before adjudication. This is a planning example, not a measured duration. Simple repositories may take less; monorepos and unresolved workflows may take substantially more.

Work in twelve batches of ten rows, or another manageable sequence. At the end of a session, save the CSV and record the reviewed IDs and any access issues. There is no need to force a supported/unsupported decision to meet a schedule: unknown is a legitimate completed judgment when its reason is documented.

Do not create a submission promise from an estimated finish date. If you decide not to complete the audit, the manuscript must continue to describe the absence of manual validation accurately.

5 Understand the evidence before assigning labels

An endpoint is one normalized GitHub repository, identified by `owner/repository`. Several package/version observations may map to the same endpoint. You must inspect all mappings listed for that endpoint when assessing the mapping criterion.

The commit SHA fixes the repository state being inspected. Open the supplied SHA-specific links, not the repository's current main branch. Later changes on GitHub must not silently

replace the sampled evidence. A missing commit in the sheet means acquisition was unresolved; do not fill in today's HEAD and treat it as the original observation.

The registry observations identify particular npm package versions. A current registry response can differ from the retained response. Prefer retained evidence for claims about the frozen acquisition and record any newly accessed material and its date separately.

The repository commit may contain a newer package version than the original npm observation. A version mismatch alone does not disprove repository mapping: the mapping criterion concerns the repository and matching package name/path. It also does not prove that the inspected commit corresponds to the published package artifact. Record the difference where relevant.

Current commit inspection cannot validate the original undated HEAD observation, historical installations, publisher authority, job execution, or intervention efficacy. The labels concern static evidence only.

All review tasks are read-only. You do not need to install dependencies, build repositories, run workflows, or execute package payloads.

6 Use exactly these three label values

- `supported`: inspected evidence satisfies the specific criterion. Cite the relevant file, location, and relationship.
- `unsupported`: inspected evidence contradicts the criterion or a sufficiently complete applicable inspection establishes its absence. Explain the contradiction or how absence was established.
- `unknown`: evidence is unavailable, incomplete, ambiguous, or cannot be resolved within the inspected material. Explain what is missing.

A blank cell means not reviewed. It is not an unknown result. Unknown may be a valid final result after inspecting an access failure or ambiguity; it is not a shortcut for a row you have not opened.

Do not interpret unknown as safe, benign, false, or unsupported. Do not infer absence from a failed request, a single missing file, or a missing item in a capped candidate-file list.

The 120 endpoints were sampled across four source-count strata, 30 per stratum, from 7,158 source-reachable endpoints. Those strata have different population sizes. An unweighted percentage over the 120 rows is a sample description, not automatically an estimate for the complete frame.

7 The five criteria: exact decision rules

7.1 Repository mapping supported

CSV field: `repository_mapping_supported`.

For every listed original package/version mapping, inspect the retained npm registry repository field and a named package manifest at the pinned GitHub commit. Check that the repository identity is consistent and that the relevant `package.json` has the expected npm package name.

In a monorepo, the root manifest may name the workspace rather than the sampled package. Follow the listed package directory, such as `packages/example/package.json`, and record the workspace relationship. A root README or similar repository name is insufficient by itself.

Use supported only when all listed mappings satisfy the criterion. If one definitely contradicts it, record unsupported and identify the mapping. If no contradiction is established but one mapping cannot be resolved, record unknown.

Normalize ordinary transport differences such as `git+https`, a trailing `.git`, and owner/repository letter case. Do not silently assume a redirect, deleted repository, ambiguous workspace path, or changed owner is equivalent without evidence.

Your note should identify every mapped package/version, the registry repository value, and the corresponding manifest path/name. This is a repository-mapping check, not a release-authenticity check.

7.2 Workflow file present

CSV field: `workflow_file_present`.

Inspect an actual workflow `.yml` or `.yaml` file under `.github/workflows/` at the pinned commit. Record its path. Merely seeing a directory name, a README badge, an action definition elsewhere, or an automation reference in documentation is insufficient.

A readable workflow file supports presence even if its job is disabled or conditional; describe those conditions separately. Presence does not establish a valid successful run.

Use unsupported for demonstrated absence in an adequately inspected pinned tree. Use unknown if access fails, the tree cannot be inspected, or the evidence is incomplete. A 404 response alone is not a sufficient general rule for absence.

7.3 Installation directive observed

CSV field: `install_directive_observed`.

Inspect workflow steps for a relevant installation instruction, such as a real `run:npmci`, `npminstall`, `pnpminstall`, or Yarn installation command, in the package path or an applicable workspace.

Follow the job's checkout path, `defaults.run.working-directory`, step-level working directory, workspace configuration, shell commands, and conditions. Record the workflow path, job, step, command and working directory. An install in an unrelated example directory does not automatically support the sampled package.

An echoed command, comment, documentation example, or setup-node step alone does not establish an installation directive. A multiline shell step needs inspection of what is actually being invoked.

For reusable workflows or local/composite actions, resolve the relevant definition and its installation step if possible. If the dependency chain, mutable external reference, or working directory cannot be resolved, use unknown. Record the uncertainty rather than guessing.

Supported means an applicable static instruction was observed. It does not mean the job triggered, completed, installed a particular transitive dependency, or accessed secrets.

7.4 Relevant lockfile present

CSV field: `lockfile_present`.

Inspect the lockfile and its relationship to the named package path. Examples include `package-lock.json`, `npm-shrinkwrap.json`, `pnpm-lock.yaml`, and `yarn.lock`.

For a workspace, determine whether the root lockfile covers the package. A lockfile in a documentation example or unrelated subproject does not qualify merely because it exists somewhere in the repository.

Record the lockfile path and how its importer/workspace/package relationship connects to the sampled package. Do not infer that the workflow used the lockfile simply because it was present.

Use unsupported only when applicable evidence establishes no relevant lockfile or clearly contradicts relevance. If candidate retrieval was capped, a file was unavailable, or workspace coverage is uncertain, use unknown.

7.5 Dependency/version example supported

CSV field: `dependency_version_supported`.

Find at least one dependency declared by the relevant manifest and its corresponding resolved version in the applicable lockfile. Record the dependency name, manifest section and declared specifier, lockfile location/key, and resolved version.

Inspect the relationship, not just two matching strings. For npm lockfiles, distinguish the root declaration from the resolved package entry. For pnpm, follow the package/workspace importer and its dependency reference to the resolved entry. For Yarn, match the dependency descriptor/specifier to its locked version. If the lockfile format is unfamiliar, resolve it before deciding or use unknown.

A transitive entry unrelated to the chosen manifest, a package version in a README, or a root package's own version is not by itself the required dependency/version example. A local workspace dependency may not supply an ordinary registry version; do not invent one.

The criterion requires a documented example, not verification of all dependencies or all transitive graph paths. Do not infer historical installation or integrity from a matching pair.

8 Repeat this workflow for each row

1. Open your own CSV and locate the next audit ID. Preserve `audit_id`, `repository` and `commit_sha` exactly.
2. Find the same ID in `Evidence_Index.html`. Confirm that the repository and pinned SHA agree with the sheet.
3. Read all original package/version mappings. Inspect the relevant retained registry observations and pinned manifests.
4. Inspect workflow files and trace applicable paths, commands and conditions.
5. Inspect relevant lockfiles and at least one dependency/version relationship if one is available.
6. Assign each of the five labels separately. Evidence missing for one criterion does not automatically decide every other criterion.
7. Write criterion-specific evidence notes, including reasons for unsupported and unknown judgments.
8. Save the sheet, then move to the next row. Record access issues and any newly accessed evidence separately.

If a supplied link fails, first check the pinned URL and retained local response. Do not replace the repository with an easier one. Failed acquisitions remain part of the sample.

The original evidence packet identifies repository response paths under `manuscript_q1/results/revision_validation/endpoint_audit/`. Copies are included in this task folder under `evidence/registry/` and `evidence/github_responses/`, with readable successful file contents under `evidence/source_text/`. The HTML index links those copies directly. Registry files contain requested package/version, observed values, repository fields, request states and timestamps. GitHub file responses contain the request metadata, pinned URL, content or error, and observation time. Open them as text; do not mistake a failed-response record for successfully retrieved source.

When sharing the task ZIP, extract the entire folder before opening the evidence index. The included retained records and source-text files can be reviewed offline. GitHub and live npm API links need internet access. Keep copied source text with its response JSON: the response identifies the commit, request, observation date and any error.

9 Fill the CSV without damaging it

Use LibreOffice Calc, Excel, another CSV editor, or a text editor that handles quoted CSV correctly. Import as UTF-8 and keep identity columns as text. Export as CSV, not an `.xlsx` file renamed to `.csv`.

Keep all 120 rows and the original header. Enter only lowercase `supported`, `unsupported`, or `unknown` in the five label columns. Do not put comments, yes/no, percentages or question marks in those cells.

Use `evidence_notes` for explanations. Spreadsheet software should quote notes containing commas or line breaks. If editing raw text, preserve valid CSV quoting; using semicolons inside one notes cell is often easier.

A useful notes structure is:

```
Mapping: [package@version -> manifest path/name; registry evidence];
Workflow: [path at pinned SHA];
Install: [job/step, command, working directory, condition];
Lock: [path and workspace/importer relationship];
Version: [dependency, declared specifier, resolved version, lock key];
Unknown/unsupported reasons: [criterion and missing/contradictory evidence].
```

The bracketed text is a template, not a result to copy literally. A note such as "checked" or "looks fine" is not an auditable explanation. Do not copy a previous endpoint's note without inspecting and updating every relevant detail.

10 Check your own sheet before sharing labels

Each author should independently confirm: all 120 IDs are present exactly once; identity columns match the supplied sheet; all five labels are filled; every row has meaningful evidence notes; unresolved evidence is described; and the CSV opens correctly after saving.

From this task folder, the optional checker can be run with:

```
python3 validate_reviews.py
```

It checks structure, allowed values and completeness, without computing agreement or revealing the other reviewer's choices. An incomplete report is expected until the work is finished. It does not validate the reasoning in your notes or prove who performed the inspection.

If one author finishes first, preserve that sheet privately and wait. Do not send the first author's labels to the second author before the second independent pass is complete.

11 Preserve independent results, then adjudicate

After both passes are complete, save dated read-only copies of both initial CSVs. Keep those initial decisions unchanged. A file checksum helps identify which bytes were used, but does not certify the truth of the judgments.

Hand back the two initial CSVs and session notes. The existing analysis program compares each label, calculates initial agreement, and creates disagreement rows. It does not require all authors to initially agree.

For each disagreement, both authors open the cited evidence together, explain their interpretation, and decide supported, unsupported or unknown under the same criterion. Record a reason and the exact evidence in the separate adjudication sheet. Do not choose the majority of two people, or prefer supported simply to improve a result. If evidence remains inconclusive, unknown is appropriate.

Do not modify initial labels to improve Cohen's kappa. Initial agreement and consensus results serve different purposes and both must be preserved.

If the process exposes a rubric defect, document it and any reassessment separately. If a third knowledgeable reviewer is needed, use an actual person, record their role and report what they did. Do not describe a discussion between the same two authors as a third independent review.

12 Technical handoff and analysis commands

You can return the completed files to the assistant and ask for integration; neither author needs to implement the analysis. If handling it yourselves, run commands from the repository root, not from this task directory.

First preserve the canonical review folder and copy the two completed task sheets into `manuscript_q1/review_packets/`, keeping the expected filenames. Do not overwrite any earlier genuine initial reviews without preserving them.

```
python3 scripts/study_validated_endpoints.py --label-source human
```

Run this after both initial sheets are complete. If they disagree, inspect the generated `manuscript_q1/review_packets/adjudication.csv`. Fill only its `adjudicated_label` and `reason` fields; preserve IDs, criterion names, and both initial labels. Rerun the command after adjudication.

If an adjudication file already exists from an earlier incomplete run, do not assume it includes every current disagreement. Ask for reconciliation against the preserved initial sheets before editing it. Do not delete completed adjudication decisions without saving a copy.

The successful human run produces `labels_adjudicated.json` and `human_review_status.json` under the endpoint-audit results, plus `human_results.csv`, `human_results.json` and `human_summary.json` under `endpoint_transport/`. Verify that `human_review_complete_and_adjudicated` is true before describing the process as complete.

The status report includes initial raw agreement, per-criterion confusion matrices and Cohen's kappa where defined. Agreement is calculated before adjudication. Kappa may be undefined when there is no category variation; report that honestly rather than replacing it with zero or one.

The numerical output must then be inspected and integrated into the manuscript. Running the command alone does not update the article, its figures, public release, or Editorial Manager.

13 What the resulting study may claim

After the actual work, the paper may describe two named authors independently applying the stated static-evidence rubric to the frozen 120-endpoint sample, preserving initial labels and resolving disagreements. The exact wording must match the real process, including any AI assistance, rubric changes, unresolved cases or incomplete rows.

Report the completed sample size, dates, criteria, unresolved counts, initial agreement and the actual adjudication procedure. There is no required agreement score in this task. Low agreement is information about the measurement, not a reason to alter records.

Use the strata and sampling weights for frame-level support estimates. Report uncertainty and unknown labels separately. Do not call an unresolved observation a false positive or a negative outcome without a justified definition.

Even after manual review, the study would establish static evidence at inspected commits. It would still not establish historical installation, actual workflow execution, publisher authorization, compromise probabilities, or security-control effectiveness. Human review does not change those underlying observation limits.

14 Record actual author contributions

Complete `templates/actual_contributions.csv` with short factual descriptions of what each author actually did. The role columns are prompts, not assignments. Leave inapplicable roles empty or mark them not applicable in your explanation.

Useful distinctions include designing the study versus running collection; developing software versus reviewing generated code; interpreting results versus checking arithmetic; drafting text versus revising its argument; and performing the endpoint audit versus reviewing its instructions.

For example, if an author performs this endpoint inspection, that actual activity can be described as validation. It does not mean that author originally implemented the collector. If software was developed with AI assistance, describe the author's real direction, checking and interpretation rather than claiming unaided coding.

Both authors should read the resulting contribution statement and correct it before it is added to the title page. First authorship, shared affiliation and availability to help do not themselves establish particular contribution roles.

15 Read and approve the actual final manuscript

Both authors should read the complete manuscript, including abstract, figures, tables, references, appendices and declarations. Open the final PDF, not only the editable source or this guide.

Check author spelling/order, departments, contacts and ORCIDs. Check that the work described

is the work actually done, that numerical summaries match their tables, that limitations remain accurate, and that AI use, funding and competing interests are described truthfully. Flag any source or interpretation you cannot support before approval.

Use the approval template to identify the exact PDF filename, version/date and SHA-256. Each author records their own actual approval and date after review. Do not prefill approval on the other person's behalf. A guide, test report or checksum cannot provide that consent.

If human validation results are later added or the manuscript changes materially, review and approve the new version. An approval of an older PDF does not automatically approve a revised one.

The latest local corrected EMSE article at task preparation is 34 pages, with 13 figures, 14 tables and a 186-word abstract. Page counts may change when real review results are incorporated; do not use those counts as substitutes for identifying the actual final file.

16 Handle the existing journal submission

The manuscript has already been submitted according to your report. Local edits, completed review sheets and a rebuilt PDF do not automatically replace files in Editorial Manager.

Use the journal's available editorial/revision route. A request draft is in `submission/emse/correction_request_draft.txt`. Only send a statement that accurately describes what is being corrected and what additional work has actually been completed. If review results are added, explain that they are additional analyses rather than describing every change as formatting.

The editorial office decides whether to return the submission for editing, accept replacement files, or ask you to wait for a formal revision. Do not create a duplicate submission simply because the original is locked.

Before approving a replacement submission PDF, check that it contains the intended manuscript and title page, correct author metadata, the matching abstract and all referenced figures. Both authors' genuine approval remains separate from clicking the portal's PDF-approval button.

17 What to return when finished

- Both original independent CSVs, unchanged after their initial completion.
- Session/access notes and any rubric clarifications or deviations.
- The adjudication CSV, if disagreements have been resolved.
- Actual contribution descriptions for both authors.
- Any corrections found while reading the manuscript.
- Both authors' approval records for the final version, only after that review is complete.

The assistant can check file structure, recompute agreement and weighted summaries, verify numerical consistency, update the manuscript and figures, rebuild the submission package, and prepare a versioned artifact update. The original public release should remain preserved; new reviewed data belong in a subsequent documented version.

The endpoint audit is finished when both real initial passes and all required adjudication are complete, not when the spreadsheet merely contains 600 nonempty label cells. Authorship approval is finished when each author has actually reviewed and agreed to the specified manuscript.

18 Source and scope

Journal policy: EMSE submission guidelines. The guidelines require author agreement and describe AI-use documentation; they do not prescribe this particular 120-endpoint audit.

Study-specific rules come from the frozen validation protocol and the original endpoint review packet in this repository. Frozen sample: `manuscript_q1/protocols/endpoint_audit_sample.json`. Canonical rubric and evidence packet: `manuscript_q1/review_packets/ENDPOINT_REVIEW_INSTRUCTIONS.md`.

This guide explains the existing task. It records no completed review, no agreement statistics, no author contribution assignments and no consent on either author's behalf.